3주차 1일차: Model·View·ViewModel의 역할 — 30분

1. 학습 목표 — 2분

- 디자인 패턴을 역할을 나누는 방법으로 이해합니다.
- 화면 상태를 MainViewModel 클래스로 옮깁니다.

시작 상태는 2주차 완료본(day08)입니다.

2. 핵심 개념 설명 — 8분

지금 MainWindow에는 화면 초기화, 입력 상태, 결과 상태, 목록, 실행 동작이 모두 들어 있습니다. 코드가 늘면 "화면 배치 수정"과 "데이터 처리 수정"이 같은 창에 모여 찾기 어려워집니다.

디자인 패턴은 이런 반복적인 구조 문제에 사용하는 역할 분담 방법입니다. MVVM은 이름 세 개를 외우기보다 현재 코드가 맡은 일을 나누는 데서 시작합니다.

역할	이번 프로그램	담당
Model	Post.cs	게시글 한 건의 데이터
View	MainWindow.xaml과 화면 관련 Code-behind	사용자에게 보여 주는 화면
ViewModel	MainViewModel.cs	화면에 제공할 입력·결과·상태·목록

오늘은 상태와 변경 알림만 MainViewModel로 옮깁니다. 버튼 처리 동작은 아직 창에 남겨 두어 한 번에 바꾸는 양을 줄입니다. 완전한 MVVM 전환은 이번 주에 단계적으로 마칩니다.

MainWindow의 DataContext를 새 ViewModel 객체로 지정하면 XAML은 그 객체의 Input·Result·Posts를 찾습니다. 속성 이름을 유지하므로 Binding 경로는 거의 그대로입니다. 파일을 나누는 것보다 **데이터를 찾는 기준 객체가 바뀐 것**이 핵심입니다.

작은 화면에서 이벤트 처리기를 쓰는 것도 자연스럽습니다. MVVM은 프로그램의 목적과 크기에 맞춰 선택하는 구조입니다.

3. 짧고 이해하기 쉬운 코드 예제 — 4분

```
private readonly MainViewModel viewModel = new();

public MainWindow()
{
    InitializeComponent();
    DataContext = viewModel;
}
```

MainViewModel은 Window를 상속하지 않는 일반 C# 클래스입니다. Input·Result·Status·Posts와 변경 알림이 그 안으로 이동합니다. 오늘은 창에서 결과를 쓸 수 있도록 Result와 Status의 set을 public으로 둡니

다.

버튼 처리기의 일부는 다음처럼 바뀝니다.

```
        viewModel.Result = text;
        viewModel.Status = "첫 행의 제목을 수정하고 새 행을 추가했습니다.";
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainViewModel.cs	day09의 MainViewModel.cs
MainWindow.xaml.cs	day09의 MainWindow.xaml.cs

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** day08이 실행되는지 확인하고 창을 닫습니다.
2. **5분:** 프로젝트에 MainViewModel.cs를 추가하고 전체 코드를 넣습니다. MainWindow.xaml.cs를 오늘의 전체 코드로 교체합니다. XAML과 Post.cs는 그대로입니다.
3. **3분:** F5로 실행해 입력 후 버튼을 누릅니다. 첫 행 변경과 새 행 추가가 이전과 똑같이 동작하는지 확인합니다.
4. **3분:** 창을 닫고 MainWindow와 MainViewModel을 나란히 엽니다. 속성·알림은 ViewModel에, 버튼 처리 순서는 창에 있다는 경계를 확인합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 8로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 9로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Model은 데이터, View는 화면, ViewModel은 화면에 제공할 상태를 맡습니다.
- MainViewModel은 일반 C# 객체이며 화면 컨트롤을 상속하지 않습니다.
- DataContext를 새 객체로 바꾸어도 같은 public 속성이 있으면 기존 바인딩을 이어 쓸 수 있습니다.
- 오늘은 전환 중간 단계입니다. 역할을 한 번에 전부 옮기지 않았습니다.

완료본: day09. 다음: 3주차 2일차.